

Programmation Web 1

Requête HTTP

Table des matières

Requête synchrone : Formulaire HTML.....	2
Requête asynchrone : AJAX.....	5
Application web monopage.....	10

Document écrit par Stéphane Gill

Novembre 2020



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

Le protocole HTTP (*Hypertext Transfer Protocol*) est un protocole de communication client-serveur développé pour le Web. Ce protocole de la couche application utilise, dans les faits, le protocole TCP comme couche de transport.

Les clients HTTP les plus connus sont les navigateurs Web permettant à un utilisateur d'accéder à un serveur contenant les données. Un serveur HTTP utilise par défaut le port 80 (443 pour HTTPS).

Dans le protocole HTTP, il existe un mécanisme pour échanger des données entre le navigateur et le serveur Web. Ce mécanisme s'appelle requête HTTP. Il existe deux types de requêtes : synchrones et asynchrones

Dans ce chapitre, nous abordons les concepts de requêtes synchrones et asynchrones.

Requête synchrone : Formulaire HTML

Les formulaires HTML sont un des vecteurs principaux d'interaction entre un utilisateur et un site web. Ils permettent à l'utilisateur d'envoyer des données au site web. La plupart du temps, ces données sont envoyées à des serveurs web mais la page peut aussi les intercepter et les utiliser elle-même.

Un formulaire HTML est composé d'un ou plusieurs items. Ceux-ci peuvent être des zones de texte (sur une seule ligne ou plusieurs lignes), des boîtes de sélection, des boutons, des cases à cocher ou des boutons radio.

Exemple 1 : Charger une page Web à l'aide d'un formulaire HTML.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="utf-8" />
5   <title>Exemple</title>
6 </head>
7
8 <body>
9   <h1>Exemple de requête synchrone</h1>
10  <form action="Informatique.html" method="post">
11    <button type="submit">Charger le fichier</button>
12  </form>
13 </body>
14 </html>
```

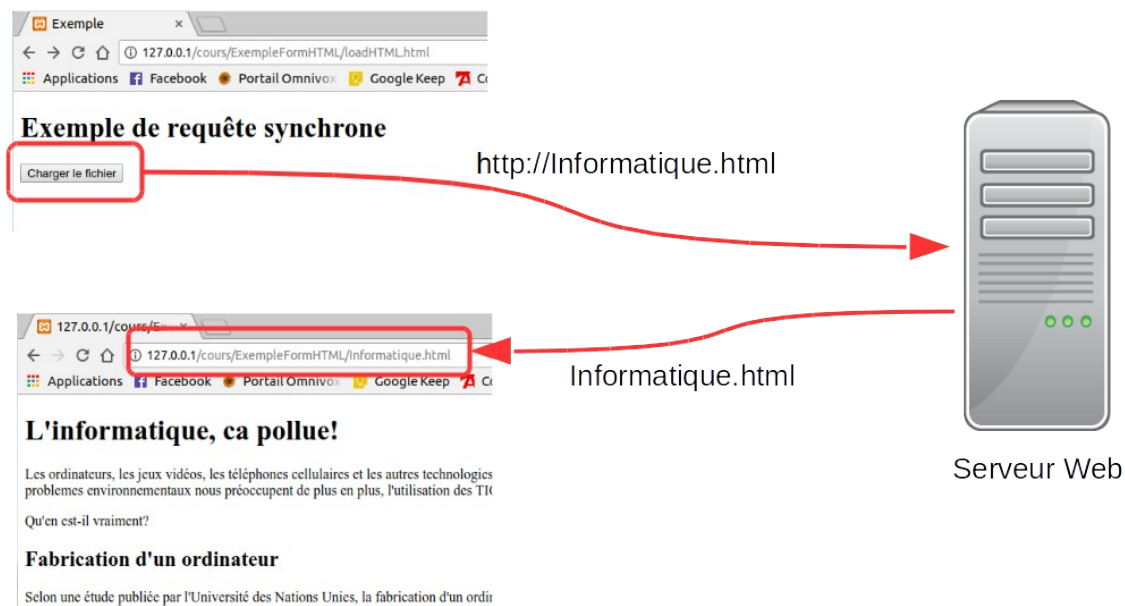
Tous les formulaires HTML sont définis par une balise `<form>` comme aux lignes 10 à 12. Tous les attributs de cette balise sont optionnels. Il est considéré comme une bonne pratique de définir au moins les attributs `action` et `method` :

- L'attribut **action** définit l'URL où doivent être envoyées les données collectées par le formulaire.

- L'attribut **method** définit la méthode HTTP utilisée pour envoyer les données (cela peut être « get » ou « post »).

La balise <button>, à la ligne 11, permet d'envoyer la requête et les données du formulaire au serveur Web. Un bouton peut être de trois types :

- **submit** envoie les données du formulaire vers la page définie par l'attribut action de l'élément <form>.
- **reset** réinitialise tous les blocs du formulaire à leurs valeurs par défaut.
- **button** ne fait rien ! Permet de concevoir des boutons personnalisés avec JavaScript.



Exemple 2 : Envoyer un message avec un formulaire HTML (POST).

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8" />
5      <title>Exemple</title>
6  </head>
7
8  <body>
9      <h1>Exemple formulaire post</h1>
10     <form action="post.php" method="post">
11         <div>
12             <label for="nom">Nom :</label>
13             <input type="text" id="nom" name="user_name" />

```

```

14     </div>
15     <div>
16         <label for="courriel">Courriel :</label>
17         <input type="email" id="courriel" name="user_email" />
18     </div>
19     <div>
20         <label for="message">Message :</label>
21         <textarea id="message" name="user_message"></textarea>
22     </div>
23     <div class="button">
24         <button type="submit">Envoyer votre message</button>
25     </div>
26 </form>
27 </body>
28 </html>

```

La balise <form> de la ligne 10 définit où et comment les données seront envoyées grâce aux attributs action et method. Il est aussi nécessaire de donner des noms aux données. Ces noms sont importants pour deux raisons. Du côté du navigateur, cela sert à déclarer quelles données doivent être envoyées. Du côté du serveur, chaque information doit avoir un nom pour être manipulée correctement. Pour nommer vos données vous devez utiliser l'attribut name pour chacun des blocs qui collectent de l'information.

Dans l'exemple précédent, le formulaire enverra trois informations nommées respectivement user_name, user_email et user_message. Ces informations seront envoyées au script post.php avec la méthode HTTP POST.

Du côté du serveur, le script php recevra les données sous forme d'une liste intégrées à une requête HTTP. Voici un exemple comme extraire les données avec un script php :

```

1 <?php
2 echo $_SERVER['REQUEST_URI'] . "<br>";
3 echo $_POST["user_name"] . "<br>";
4 echo $_POST["user_email"] . "<br>";
5 echo $_POST["user_message"];
6 ?>

```

La méthode post envoie un en-tête et un corps de message au serveur. Le corps est généralement constitué des données entrées dans le champ de formulaire par l'utilisateur. Les données du formulaire n'apparaissent pas dans l'URL.

Exemple 3 : Envoyer un message avec un formulaire HTML (GET).

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <meta charset="utf-8" />
5     <title>Exemple</title>
6 </head>
7

```

```

8 <body>
9   <h1>Exemple formulaire post</h1>
10  <form action="get.php" method="get">
11    <div>
12      <label for="nom">Nom :</label>
13      <input type="text" id="nom" name="user_name" />
14    </div>
15    <div>
16      <label for="courriel">Courriel :</label>
17      <input type="email" id="courriel" name="user_email" />
18    </div>
19    <div>
20      <label for="message">Message :</label>
21      <textarea id="message" name="user_message"></textarea>
22    </div>
23    <div class="button">
24      <button type="submit">Envoyer votre message</button>
25    </div>
26  </form>
27 </body>
28 </html>

```

Du côté du serveur, le script `get.php` recevra les données sous forme d'une liste intégrées à une requête HTTP. Voici un exemple comme extraire les données avec un script php :

```

1 <?php
2 echo $_SERVER['REQUEST_URI'] . "<br>";
3 echo $_GET["user_name"] . "<br>";
4 echo $_GET["user_email"] . "<br>";
5 echo $_GET["user_message"];
6 ?>

```

La méthode `get` encode les données du formulaire dans l'URL. Celle-ci est composée du nom de la page ou du script à charger avec les données de formulaire empaquetée dans une chaîne. Les données sont séparées de l'adresse de la page par le code `?` et entre elles par le code `&`.

Voici un exemple d'URL utilisant la méthode `GET` :

`http://127.0.0.1/get.php?user_name=allo&user_email=allo%40gmail.com&user_message=Bonjour%21`

Requête asynchrone : AJAX

Dans une application Web, la méthode classique de dialogue entre un navigateur et un serveur est la suivante : lors de chaque manipulation faite par l'utilisateur, le navigateur envoie une requête contenant une référence à une page Web, puis le serveur Web effectue des calculs, et envoie le résultat sous forme d'une page Web à destination du navigateur. Celui-ci affichera alors la page qu'il vient de recevoir. Chaque manipulation entraîne la transmission et l'affichage d'une nouvelle page. L'utilisateur doit attendre l'arrivée de la réponse pour effectuer d'autres manipulations.

En utilisant *AJAX*, le dialogue entre le navigateur et le serveur se déroule la plupart du temps de la manière suivante : un programme écrit en langage de programmation *JavaScript*, incorporé dans une page web, est exécuté par le navigateur. Celui-ci envoie en arrière-plan des demandes au serveur Web, puis modifie le contenu de la page actuellement affichée par le navigateur Web en fonction du résultat reçu du serveur, évitant ainsi la transmission et l'affichage d'une nouvelle page complète.

La méthode classique de dialogue utilise des mécanismes propres au *World Wide Web*, qui sont incorporés dans tous les navigateurs ainsi que les robots d'indexation, et ne nécessite pas de programmation. Au contraire, le fonctionnement d'*AJAX* nécessite de programmer en *JavaScript* les échanges entre le navigateur et le serveur Web. Il nécessite également de programmer les modifications à effectuer dans la page Web à la réception des réponses, sans quoi les dialogues se font à l'insu de l'utilisateur.

En *AJAX*, comme le nom l'indique, les demandes sont effectuées de manière asynchrone : le navigateur Web continue d'exécuter le programme *JavaScript* alors que la demande est partie, il n'attend pas la réponse envoyée par le serveur Web et l'utilisateur peut continuer à effectuer des manipulations pendant ce temps.

Exemple 4 : Charger un texte dans une page Web (loadHTML.html)

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Exemple AJAX</h1>
5
6  <div id="article"><p>Charger le texte ICI</p></div>
7
8  <button type="button" onclick="loadDoc()">Charger le texte</button>
9
10 <script>
11 function loadDoc() {
12     fetch("Informatique.html")
13     .then(function(reponse){
14         // Lire le flux (stream) reponse jusqu'au bout.
15         return reponse.text();
16     })
17     .then(function(html){
18         document.getElementById("article").innerHTML = html;
19     });
20 }
21 </script>
22
23 </body>
24 </html>
```

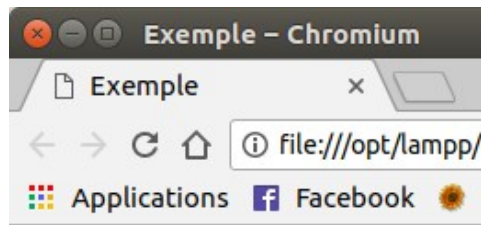
L'interface de programmation d'application (API) Fetch fournit des outils pour effectuer des requêtes HTTP. L'utilisation la plus simple de la méthode `fetch()` prend un seul paramètre, soit l'URL de la

ressource à accéder. Cette méthode retourne une promesse (promise) contenant un objet de type Response. L'instruction .then() permet d'attendre la promesse.



Exemple 5 : Charger la météo dans une page Web.

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5    <meta charset="utf-8" />
6    <title>Exemple</title>
7  </head>
8
9  <body>
10 <h1>Exemple AJAX</h1>
11
12 <div id="meteo"><p>Charger la météo ICI</p></div>
13
14 <button type="button" onclick="loadMeteo()">Charger</button>
15
16 <script>
17 function loadMeteo() {
18   let api_key = 'xxxxxxxxxx';
19   let url = 'http://api.openweathermap.org/data/2.5/weather?q=Montreal,
20             ca&mode=html&units=metric&appid=' + api_key;
21   fetch(url)
22     .then(reponse => reponse.text())
23     .then(function(html){
24       document.getElementById("meteo").innerHTML = html;
25     });
26 }
27 </script>
28
29 </body>
30 </html>
```



Exemple AJAX



Exemple 6 : Obtenir des indices

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Exemple AJAX</title>
5   <script src="Ajax4.js"></script>
6 </head>
7 <body>
8   <h1>Exemple AJAX</h1>
9   <h3>Saisir un nom</h3>
10
11   <form action="">
12     Nom: <input type="text" id="txt1" onkeyup="affIndices(this.value)">
13   </form>
14
15   <p>Suggestions: <span id="txt2"></span></p>
16 </body>
17 </html>
```

Note : L'élément HTML `<span>` est un conteneur générique en ligne pour les contenus phrasés. Il ne représente rien de particulier. Il peut être utilisé pour grouper des éléments afin de les mettre en forme. Il doit uniquement être utilisé lorsqu'aucun autre élément sémantique n'est approprié. `<span>` est très

proche de l'élément <div>, mais l'élément <div> est un élément de bloc, alors que est un élément en ligne.

```
1 function affIndice(str) {
2
3     if (str.length == 0) {
4         document.getElementById("txt2").innerHTML = "";
5         return;
6     }
7     fetch("getIndice.php?q="+str)
8     .then(reponse => reponse.text())
9     .then(texte => {document.getElementById("txt2").innerHTML = texte})
10 }
```

```
1 <?php
2 $noms[] = "Luis";
3 $noms[] = "Yanick";
4 $noms[] = "Maxym";
5 $noms[] = "Jeremie";
6 $noms[] = "Byron-Manuel";
7 $noms[] = "Merouane";
8 $noms[] = "Charl-Alexandre";
9 $noms[] = "Charles";
10 $noms[] = "Louis-Philippe";
11 $noms[] = "Laurent";
12 $noms[] = "Etienne";
13 $noms[] = "Nathalie";
14 $noms[] = "Stephane";
15
16 $q = $_GET["q"];
17
18 $indice = "";
19
20 if ($q !== "") {
21     $q = strtolower($q);
22     $len=strlen($q);
23     foreach($noms as $nom) {
24         if (stristr($q, substr($nom, 0, $len))) {
25             if ($indice === "") {
26                 $indice = $nom;
27             } else {
28                 $indice .= ", $nom";
29             }
30         }
31     }
32 }
33
34 echo $indice === "" ? "Pas de suggestion" : $indice;
35 ?>
```

Application web monopage

Une application web monopage (en anglais *single-page application* ou *SPA*) est une application web accessible via une page web unique. Le but est d'éviter le chargement d'une nouvelle page à chaque action demandée, et de fluidifier ainsi l'expérience utilisateur. Deux méthodes existent pour ce faire :

- L'ensemble des éléments de l'application est chargé (contenu, images, CSS et JavaScript) dans un unique fichier HTML,
- Soit les ressources nécessaires sont récupérées et affichées dynamiquement en fonction des actions de l'utilisateur.

L'enregistrement en local de la page définissant une application web monopage et la possibilité de continuer à l'exécuter en local est l'une des propriétés importantes des applications web monopage qui les distingue des applications web standards qui reposent sur l'existence d'un serveur HTTP avec lequel elles échangent données, continuations applicatives et interfaces.

Quand les applications web monopages gèrent des données et permettent de les modifier, pour conserver ces données modifiées, il est nécessaire que ces applications modifient leur code, c'est-à-dire : elles doivent être capables de se modifier pour que la sauvegarde locale de l'état modifié de l'application (dont les données) soit persistant.

Exemple 7 : Site Web monopage

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8" />
5      <link rel="stylesheet" type="text/css" href="style.css" media="all" >
6      <script src="script.js"></script>
7      <title>Site Web monopage</title>
8    </head>
9    <body>
10     <header>
11       <h1>Site Web monopage</h1>
12     </header>
13
14     <nav>
15       <h1>Menu</h1>
16       <ul>
17         <li><a href="#" onclick="loadDoc('page1.html')">Page 1</a></li>
18         <li><a href="#" onclick="loadDoc('page2.html')">Page 2</a></li>
19       </ul>
20     </nav>
21
22     <section>
23       <article id="leTexteIci">
24         <p><br/><br/><br/>Insérer le texte ici<br/><br/><br/></p>
```

```
25     </article>
26 </section>
27
28 <footer>
29     <p>Copyright Le Prof - Tous droits réservés<br />
30 </footer>
31
32 </body>
33 </html>
```

```
1 function loadDoc(url) {
2     fetch(url)
3     .then(function(reponse){
4         return response.text();
5     })
6     .then(function(html){
7         document.getElementById("leTexteIci").innerHTML = html;
8     });
9 }
```